Taller: Ciclo de Vida del Desarrollo de Software (SDLC)

Jhon Anderson Solano Holguin

Ingeniería de Software

Julio Andrés Silva Aragón

Corporación Universitaria Remington – Sede Yopal

29 de octubre de 2026

Contenido

Resumen 4

Abstract 5

Objetivos 6

Objetivo general 6

Objetivos específicos 6

Introducción 7

1. Fase de planeación 7

2. Fase de análisis de requisitos 12

Historias de usuario y criterios de aceptación 15

3. Fase de diseño 16

Diagrama de casos de uso 18

Diagrama de secuencia: reservar producto 19

Prototipo textual de las tres pantallas 20

4. Fase de implementación 21

5. Fase de pruebas 22

Código de pruebas unitarias JUnit 5 25

Matriz de trazabilidad 26

6. Fase de despliegue 27

Borrador de acta de entrega 28

7. Fase de mantenimiento 29

Conclusiones 31

Referencias 32

Resumen

El presente trabajo desarrolla la aplicación del Ciclo de Vida del Desarrollo de Software (SDLC) mediante el análisis y diseño de una solución tecnológica para el proyecto Mercado Campesino Digital de Yopal. A lo largo del documento se abordan las fases de planeación, análisis de requisitos, diseño, implementación, pruebas, despliegue y mantenimiento, elaborando los entregables correspondientes para cada una de ellas. Asimismo, se identifican los requisitos funcionales y no funcionales del sistema, se propone una arquitectura adecuada, se diseñan casos de prueba y una matriz de trazabilidad, garantizando la relación entre los requisitos, el diseño y la validación del software. Finalmente, se selecciona un modelo de ciclo de vida acorde con las características del proyecto, considerando las restricciones de tiempo, presupuesto y conectividad, con el propósito de desarrollar una solución de software que contribuya a mejorar la gestión de pedidos y el acceso a la información por parte de productores, clientes y la Secretaría de Desarrollo Económico de Yopal.

***Palabras clave*:** ciclo de vida del software, SDLC, ingeniería de software, requisitos, trazabilidad, Mercado Campesino Digital

Abstract

This paper presents the application of the Software Development Life Cycle (SDLC) through the analysis and design of a technological solution for the Mercado Campesino Digital de Yopal project. The study addresses the stages of planning, requirements analysis, design, implementation, testing, deployment, and maintenance, producing the corresponding deliverables for each phase. In addition, functional and non-functional requirements are identified, an appropriate software architecture is proposed, test cases and a traceability matrix are developed to ensure consistency between requirements, design, and software validation. Finally, a suitable software development life cycle model is selected according to the project's characteristics, considering constraints such as time, budget, and connectivity, with the aim of providing an efficient solution that improves order management and information access for producers, customers, and the Municipal Secretariat of Economic Development.

Keywords: software development life cycle, SDLC, software engineering, requirements, traceability, digital marketplace.

Objetivos

Objetivo general

Desarrollar una propuesta de solución de software para el proyecto Mercado Campesino Digital de Yopal mediante la aplicación de las fases del Ciclo de Vida del Desarrollo de Software (SDLC), con el fin de optimizar la gestión de la oferta de productos, la reserva por parte de los clientes y la generación de reportes de ventas, cumpliendo las restricciones de tiempo, presupuesto y conectividad establecidas para el Proyecto.

Objetivos específicos

- Analizar las necesidades de los productores, clientes y la Secretaría de Desarrollo Económico para identificar los requisitos funcionales y no funcionales del sistema.
- Diseñar la arquitectura del sistema, el modelo de datos, los diagramas UML y los prototipos de interfaz que permitan representar la estructura y funcionamiento de la plataforma.
- Elaborar casos de prueba, pruebas unitarias y una matriz de trazabilidad que permitan verificar el cumplimiento de los requisitos y asegurar la calidad de la solución de software antes de su despliegue y mantenimiento.

Introducción

El ciclo de vida del desarrollo de software permite organizar el trabajo desde la identificación de una necesidad hasta la operación y mejora de la solución. El taller propone aplicar sus siete fases a un caso local: el Mercado Campesino Digital de Yopal, donde actualmente los pedidos se reciben por WhatsApp y la información de ventas se encuentra dispersa.

El propósito de este informe es diseñar una solución viable dentro de cuatro meses y un presupuesto de $25.000.000 COP, considerando a productores con experiencia digital limitada, conectividad móvil intermitente y las obligaciones aplicables al tratamiento de datos personales. Las decisiones de alcance y diseño se presentan como una propuesta académica que deberá validarse con los interesados antes de su implementación real.

1. Fase de planeación

Problema. La gestión de la oferta y de los pedidos del Mercado Campesino de Yopal mediante conversaciones individuales de WhatsApp dificulta conocer la disponibilidad de productos, ocasiona excedentes o faltantes y no produce estadísticas consolidadas para apoyar las decisiones de la Secretaría.

Objetivo general. Diseñar e implementar en cuatro meses una plataforma web responsiva que permita a los productores publicar su oferta semanal, a los clientes reservar productos antes del viernes a las 6:00 p. m. y a la Secretaría consultar reportes, logrando que al menos el 90 % de las reservas de la prueba piloto se registren sin intervención manual del equipo técnico.

Objetivos específicos: (a) levantar y validar los requisitos con productores, clientes y Secretaría durante el primer mes; (b) construir un catálogo y un módulo de ofertas que puedan utilizarse desde celulares Android de gama baja; (c) implementar reservas con validación de disponibilidad y cierre automático en el horario establecido; (d) entregar reportes por producto, productor y vereda, y capacitar a los usuarios antes del lanzamiento.

Alcance. La primera versión se concentra en publicación de ofertas, reservas y reportes. Los domicilios se excluyen expresamente para no comprometer plazo ni presupuesto; si se aprueban en 2027, deberán tratarse como una solicitud de cambio, estimarse y recorrer nuevamente las fases pertinentes del SDLC.

Factibilidad técnica. Es viable desarrollar una aplicación web responsiva con una API y una base de datos relacional. La operación con señal intermitente se atenderá mediante pantallas livianas, guardado de formularios cuando sea posible y mensajes claros ante pérdida de conexión. No se promete operación completamente sin conexión en la primera versión; esa capacidad requiere pruebas y alcance específico.

Factibilidad económica. El presupuesto de $25.000.000 COP es un límite de inversión. Se recomienda priorizar funciones esenciales, usar tecnologías de código abierto y reservar contingencia. La estimación de costos de alojamiento presentada en despliegue es orientativa y debe confirmarse con cotizaciones antes de contratar.

Factibilidad operativa. El diseño debe reducir la cantidad de pasos, usar lenguaje cotidiano, botones grandes y formularios breves. Se realizarán pruebas de usabilidad con productores representativos y capacitación práctica basada en tareas reales.

Modelo seleccionado. Se propone un modelo incremental con iteraciones ágiles cortas. Primero se valida el catálogo y la publicación de ofertas; luego las reservas; finalmente los

reportes, la estabilización y el lanzamiento. Este enfoque permite recibir retroalimentación temprana y ajustar la interfaz. Se descarta cascada porque la usabilidad y los detalles operativos deben validarse con usuarios reales. Se descarta el modelo en V porque su énfasis en correspondencia formal entre fases de especificación y pruebas añade rigidez para un producto cuyo alcance requiere aprendizaje continuo. La verificación y las pruebas sí se conservan dentro del modelo incremental.

Tabla 1

Alcance del proyecto Mercado Campesino Digital de Yopal

Elemento	Incluye en la versión inicial	No incluye en la versión inicial
Oferta	Registro de productores; publicación semanal de producto, cantidad, precio y unidad; edición y desactivación de ofertas.	Gestión integral de inventarios agrícolas o integración con sistemas externos.
Reservas	Reserva previa al viernes a las 6:00 p. m.; confirmación; consulta de reservas por productor.	Domicilios, rutas de reparto y seguimiento de entregas.
Pagos	Pago contra entrega en efectivo o Nequi, acordado entre cliente y productor.	Procesamiento de pagos en línea o custodia de dinero.
Reportes	Resumen por producto, productor y vereda para la Secretaría.	Analítica predictiva avanzada o integración contable.
Plataforma	Aplicación web responsiva, autenticación y controles de acceso.	Aplicación móvil nativa y funcionamiento totalmente sin conexión.

Nota. Elaboración propia con base en el caso propuesto del taller sobre el Mercado Campesino Digital de Yopal.

Tabla 2

Matriz de riesgos del Proyecto

Riesgo	Prob. (1–5)	Impacto (1–5)	P×I	Mitigación
R-01. Señal móvil intermitente impide completar formularios.	4	4	16	Diseñar pantallas livianas, validar conexión, conservar temporalmente los datos del formulario cuando sea seguro y probar en zonas de señal débil.
R-02. Productores encuentran difícil usar la plataforma.	4	4	16	Pruebas de usabilidad tempranas, botones grandes, lenguaje simple, guías visuales y acompañamiento presencial.
R-03. Dos clientes reservan simultáneamente la última unidad.	3	5	15	Validar y descontar disponibilidad en una transacción atómica; bloquear sobreventa y probar concurrencia.
R-04. El alcance crece por la	4	4	16	Registrar domicilios como fuera de alcance;

solicitud de domicilios.				exigir solicitud de cambio con costo, plazo, riesgos y aprobación.
R-05. Exposición indebida de datos personales.	2	5	10	Minimización de datos, control de acceso, aviso de privacidad, autorización cuando corresponda, registro de accesos y medidas de seguridad.
R-06. Retraso por equipo de solo tres desarrolladores.	3	4	12	Priorizar Must, estimar por iteración, integrar continuamente y reservar las últimas dos semanas para estabilización.

Nota. Elaboración propia a partir del análisis de riesgos realizado para el proyecto.

Tabla 3

Cronograma general de desarrollo del proyecto

Mes	Actividades e hitos	Entregables
1	Planeación, entrevistas, requisitos, mapa de procesos y prototipos iniciales.	Acta del proyecto, alcance, riesgos, requisitos priorizados y prototipo validado.
2	Arquitectura, base de datos, autenticación, catálogo y publicación de ofertas.	Diseño técnico y primer incremento funcional.
3	Reservas, control de disponibilidad, cierre de	Segundo incremento, pruebas unitarias e integración.

	reservas y reportes preliminares.	
4	Pruebas de aceptación, correcciones, capacitación, piloto, despliegue y entrega.	Versión candidata, manual, evidencias de prueba y acta de entrega.

Nota. Elaboración propia según la planificación propuesta para las fases del SDLC.

2. Fase de análisis de requisitos

Los requisitos se identifican con códigos únicos para mantener trazabilidad durante diseño, implementación y pruebas. La prioridad MoSCoW clasifica las funciones como Must (imprescindible), Should (importante), Could (deseable) y Won't en esta versión (fuera del alcance actual). Los valores medibles que siguen son criterios propuestos y deben validarse con los interesados.

Tabla 4

Identificación de interesados y sus necesidades

Interesado	Necesidad o expectativa
Productores	Publicar y actualizar productos, cantidades, precios y unidades con pocos pasos; consultar reservas recibidas.
Clientes	Consultar la oferta semanal, reservar antes del cierre y recibir confirmación clara.
Secretaría de Desarrollo Económico	Consultar cifras consolidadas por producto, productor y vereda.
Administrador de plataforma	Gestionar usuarios, roles y reportes de incidentes sin acceder a más datos de los necesarios.
Equipo de desarrollo y soporte	Contar con requisitos verificables, registro de errores, despliegue controlado y documentación.
Responsable del tratamiento de datos	Informar finalidades, limitar acceso y atender solicitudes relacionadas con datos personales.

Nota. Elaboración propia con base en los actores involucrados en el proyecto.

Tabla 5

Requisitos funcionales del Sistema

Código	Requisito funcional	Prioridad
RF-01	El sistema debe permitir registrar productores con nombre, contacto y vereda, validando los campos obligatorios.	Must
RF-02	El sistema debe permitir que un productor autenticado cree, edite y desactive una oferta semanal con producto, cantidad, precio y unidad.	Must
RF-03	El sistema debe mostrar a los clientes las ofertas activas y su disponibilidad por semana.	Must
RF-04	El sistema debe permitir a un cliente registrar una reserva antes del viernes a las 6:00 p. m.	Must
RF-05	El sistema debe validar la disponibilidad y evitar que las reservas confirmadas superen la cantidad publicada.	Must
RF-06	El sistema debe generar una confirmación con número de reserva, productos, cantidades y datos de recogida.	Must
RF-07	El sistema debe permitir al productor consultar sus reservas y marcar su estado de preparación o entrega.	Should
RF-08	El sistema debe generar reportes por producto, productor y vereda, con filtro por fecha o jornada.	Must
RF-09	El sistema debe permitir a un administrador crear,	Must

	desactivar y asignar roles a usuarios.	
RF-10	El sistema debe permitir exportar los reportes consolidados a CSV o PDF.	Could
['RF-11', 'El sistema debe registrar fecha y hora de creación y cambios relevantes de las ofertas y reservas.', 'Should']		

Nota. Elaboración propia utilizando la metodología MoSCoW para la priorización de requisitos.

Tabla 6

Requisitos no funcionales asociados a ISO/IEC 25010

Código	Requisito no funcional medible	Atributo ISO/IEC 25010
RNF-01	Con una carga de 100 usuarios concurrentes y datos de prueba representativos, el 95 % de las consultas del catálogo debe responder en 3 segundos o menos.	Eficiencia de desempeño
RNF-02	Todas las funciones de gestión de ofertas y reservas deben requerir autenticación; los permisos deben verificarse en el servidor y las contraseñas no deben almacenarse en texto plano.	Seguridad
RNF-03	Al menos 4 de cada 5 productores participantes deben publicar una oferta de prueba sin ayuda después de una capacitación máxima de 20 minutos.	Usabilidad
RNF-04	Las pantallas principales deben funcionar en las dos versiones recientes de Chrome para Android y en	Compatibilidad

	resoluciones desde 360 píxeles de ancho.	
RNF-05	El sistema debe recopilar solo los datos personales necesarios, informar las finalidades y aplicar controles de acceso y procedimientos para atender derechos de los titulares conforme a la Ley 1581 de 2012 y normas reglamentarias aplicables.	Seguridad y privacidad por diseño
RNF-06	Las páginas de catálogo y publicación no deben superar 1 MB de recursos transferidos en la carga inicial bajo la configuración de producción, sin contar imágenes opcionales.	Eficiencia de desempeño
RNF-07	El sistema debe registrar errores técnicos con fecha, componente y código de correlación, evitando guardar contraseñas o datos sensibles en los registros.	Mantenibilidad y seguridad

Nota. Elaboración propia con base en los atributos de calidad definidos por la norma ISO/IEC 25010.

Historias de usuario y criterios de aceptación

HU-01 — Productor. Como productor del Mercado Campesino, quiero publicar la oferta de la semana para que los clientes conozcan los productos disponibles antes de asistir a la feria.

Dado que el productor inició sesión y completó producto, cantidad, precio y unidad, cuando selecciona “Publicar”, entonces la oferta queda activa y aparece en el catálogo. Dado que falta un campo obligatorio o la cantidad no es positiva, cuando intenta publicar, entonces el sistema indica el error y no guarda la oferta.

HU-02 — Cliente. Como cliente de Yopal, quiero reservar productos antes del cierre semanal para recogerlos el sábado en el puesto del productor.

Dado que hay unidades disponibles y la reserva se realiza antes del viernes a las 6:00 p. m., cuando el cliente confirma la cantidad, entonces el sistema registra la reserva y reduce la disponibilidad en una operación atómica. Dado que la cantidad solicitada supera la disponibilidad o ya pasó la hora límite, cuando intenta reservar, entonces el sistema rechaza la operación y explica el motivo.

HU-03 — Secretaría. Como funcionario autorizado de la Secretaría, quiero consultar reportes por producto, productor y vereda para evaluar los resultados del programa.

Dado que el funcionario inició sesión con rol autorizado, cuando selecciona una jornada y solicita el reporte, entonces el sistema muestra totales agrupados por producto, productor y vereda. Dado que un usuario sin el rol requerido intenta consultar el reporte, cuando realiza la solicitud, entonces el sistema deniega el acceso y registra el evento.

3. Fase de diseño

Arquitectura propuesta. Se plantea una arquitectura web de tres capas: (1) presentación: HTML, CSS y JavaScript responsivo; (2) lógica de negocio: API REST implementada en Java con Spring Boot; y (3) persistencia: base de datos relacional PostgreSQL o MySQL. La separación facilita probar reglas de negocio, mantener la interfaz y proteger las operaciones en el servidor. La selección definitiva del motor de base de datos dependerá de la infraestructura disponible.

Seguridad y privacidad. Se aplicará control de acceso por roles (productor, cliente, funcionario y administrador), comunicación HTTPS, validación del lado del servidor, copias de seguridad y minimización de datos. Antes del uso real deben definirse el responsable y los encargados del tratamiento, el aviso de privacidad, las autorizaciones aplicables, el periodo de

conservación y el procedimiento para consultas o reclamos conforme al marco colombiano de protección de datos.

Regla de concurrencia. La disponibilidad no debe comprobarse únicamente en la interfaz. El servidor debe comprobarla y registrar la reserva dentro de una transacción; si dos clientes intentan reservar la última unidad, solo una operación puede confirmar el descuento. Si la segunda operación encuentra disponibilidad insuficiente, se rechaza sin alterar el inventario.

Tabla 7

Modelo de entidades y relaciones del sistema

Entidad	Atributos clave	Relaciones y cardinalidad
Usuario	id, nombre, correo/teléfono, hash_contrasena, rol, estado	Un usuario puede ser productor; un usuario puede crear muchas reservas.
Productor	id, usuario_id, nombre_publico, vereda, contacto	Un productor publica muchas ofertas (1:N).
Producto	id, nombre, categoría, unidad_base, estado	Un producto aparece en muchas ofertas (1:N).
Oferta	id, productor_id, producto_id, semana, cantidad_publicada, cantidad_disponible, precio, estado	Cada oferta pertenece a un productor y a un producto; una oferta puede estar en muchas líneas de reserva.
Reserva	id, cliente_id, fecha_hora, estado, codigo_confirmacion	Un cliente crea muchas reservas (1:N); una reserva contiene muchas líneas.
DetalleReserva	id, reserva_id, oferta_id, cantidad, precio_unitario	Relaciona Reserva y Oferta; conserva precio y cantidad confirmados.
Auditoria	id, usuario_id, evento, fecha_hora, referencia	Un usuario puede generar muchos eventos de auditoría; no almacenar secretos.

Nota. Elaboración propia para representar la estructura de datos del sistema propuesto.

Tabla 8

Componentes y responsabilidades de la arquitectura del sistema

Componente	Responsabilidad
Interfaz web responsiva	Presentar catálogo, formularios de oferta, reserva y reportes con navegación simple.
API REST	Autenticar, autorizar y exponer operaciones; validar reglas y formatos.
Servicio de reservas	Verificar fecha límite y disponibilidad, crear reserva y descontar cantidades transaccionalmente.
Repositorio de datos	Persistir usuarios, ofertas, reservas y auditoría con restricciones de integridad.
Servicio de reportes	Agrupar reservas y cantidades por producto, productor, vereda y jornada.
Infraestructura	HTTPS, copias de seguridad, monitoreo, registros técnicos y restauración.

Nota. Elaboración propia según la arquitectura propuesta para la solución de software.

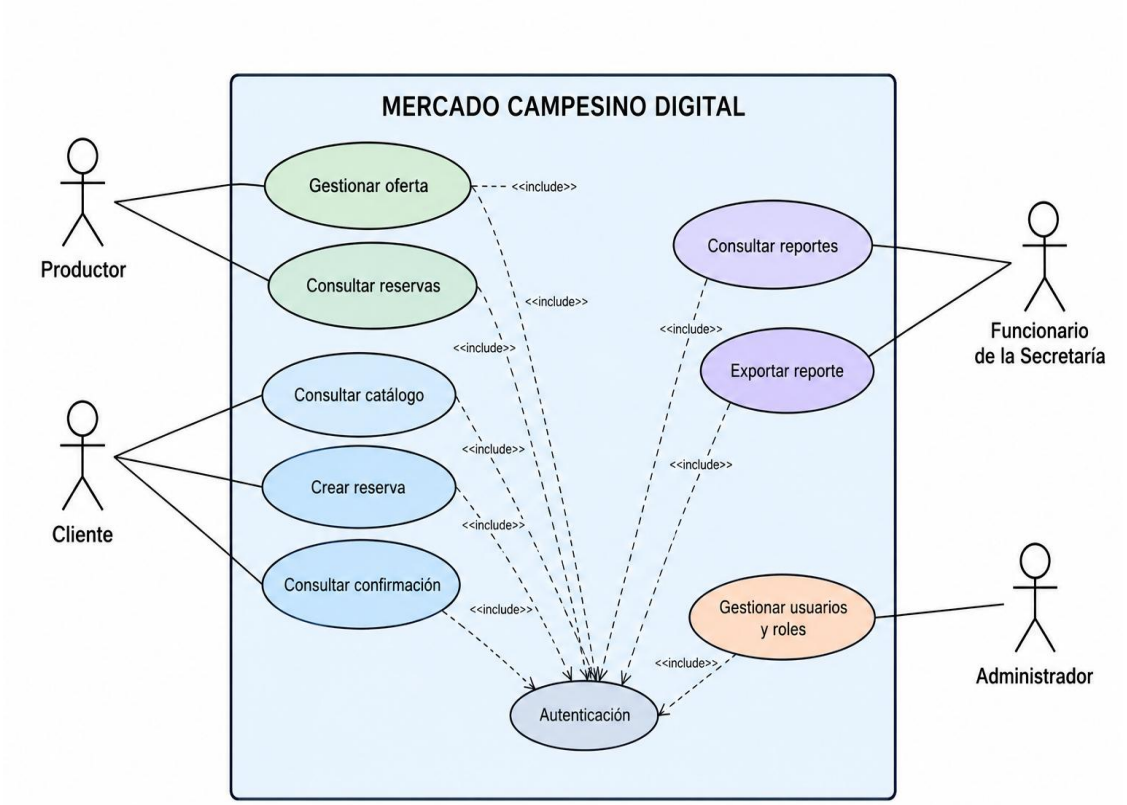
Diagrama de casos de uso

Actores: Productor, Cliente, Funcionario de la Secretaría y Administrador.

Casos de uso principales: Productor → iniciar sesión, gestionar oferta, consultar reservas y actualizar estado de preparación/entrega. Cliente → consultar catálogo, crear reserva y consultar confirmación. Funcionario → consultar y exportar reportes. Administrador → gestionar usuarios y roles. Todos los casos protegidos requieren autenticación; la API autoriza cada acción según el rol.

Figura 1

Diagrama de casos de uso del sistema Mercado Campesino Digital



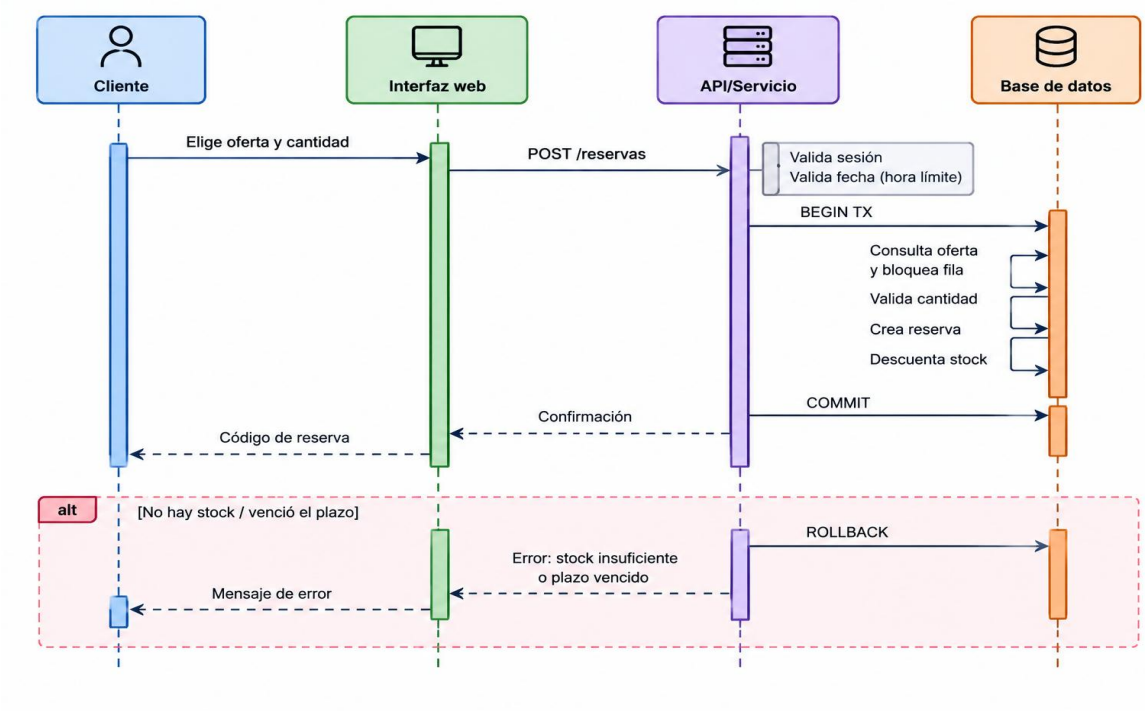
Nota**.** Elaboración propia. El diagrama representa los actores principales (Productor, Cliente, Funcionario de la Secretaría y Administrador) y los casos de uso asociados al sistema Mercado Campesino Digital, incluyendo el proceso de autenticación y la autorización de acciones según el rol de cada usuario.

Diagrama de secuencia: reservar producto

El flujo valida primero la hora límite y la disponibilidad en el servidor. La reserva y la actualización de cantidad deben confirmarse en una misma transacción.

Figura 2

Diagrama de secuencia para el proceso de reserva de productos



Nota. Elaboración propia. El diagrama representa la interacción entre el cliente, la interfaz web, la API o servicio y la base de datos durante el proceso de reserva de un producto. Asimismo, muestra la validación de la sesión, la disponibilidad del inventario, el manejo de transacciones (BEGIN, COMMIT y ROLLBACK) y la respuesta del sistema cuando la reserva se realiza correctamente o cuando no existe disponibilidad de stock o ha vencido el plazo de reserva.

Prototipo textual de las tres pantallas

Pantalla 1 — Publicar oferta (productor): encabezado “Mi oferta para el sábado”; campos Producto, Cantidad disponible, Precio, Unidad y Semana; botón principal “Publicar oferta”; botón secundario “Guardar borrador”; aviso visible sobre la fecha de cierre. Los errores se muestran junto al campo correspondiente.

Pantalla 2 — Reservar (cliente): catálogo en tarjetas sencillas con nombre, precio, unidad, productor, vereda y disponibilidad; control para elegir cantidad; resumen de la reserva; aviso "Reservas hasta el viernes, 6:00 p. m."; botón "Confirmar reserva". La confirmación muestra código, productos, cantidades y lugar de recogida.

Pantalla 3 — Reportes (Secretaría): filtros por jornada, producto, productor y vereda; indicadores de reservas y cantidades; tabla de resultados y opción de exportar. El acceso se restringe a funcionarios autorizados. Estos prototipos son especificaciones funcionales; se recomienda convertirlos en wireframes visuales antes de codificar.

4. Fase de implementación

Repositorio. Se recomienda crear un repositorio GitHub privado durante el desarrollo y público solo si la institución y los responsables autorizan la publicación del código. Estructura sugerida: /src/main/java, /src/main/resources, /src/test/java, /docs, /sql y README.md. El README debe explicar objetivo, requisitos previos, configuración, ejecución, pruebas y despliegue.

Estrategia de ramas. main contiene versiones estables; develop integra cambios; feature/nombre-funcionalidad se usa para cada tarea. Cada cambio se incorpora mediante pull request y revisión de otro integrante. Se recomienda al menos un commit significativo por integrante, según las condiciones del taller.

La siguiente regla Java ilustra la validación de la reserva. Para una implementación real, la lectura y el descuento deben ejecutarse en una transacción con bloqueo optimista o pesimista en la base de datos; el ejemplo de dominio no sustituye ese control de concurrencia.

```
import java.time.DayOfWeek;
import java.time.LocalDateTime;
import java.time.LocalTime;
```

```
public class ReglaReserva {
    private static final LocalTime CIERRE = LocalTime.of(18,
0);

    /**
     * RF-04 y RF-05: valida plazo y disponibilidad.
     * La persistencia debe descontar cantidad dentro de una
transacción.
     */
    public int validarReserva(LocalDateTime ahora,
                              int cantidadSolicitada,
                              int cantidadDisponible) {
        if (ahora == null) {
            throw new IllegalArgumentException("La fecha es
obligatoria");
        }
        // Cierre: viernes a las 18:00; una vez alcanzado, no
se aceptan reservas.
        boolean despuesDelCierre =
            ahora.getDayOfWeek() == DayOfWeek.FRIDAY
            && !ahora.toLocalTime().isBefore(CIERRE);
        boolean diaPosterior =
            ahora.getDayOfWeek() == DayOfWeek.SATURDAY
            || ahora.getDayOfWeek() == DayOfWeek.SUNDAY;
        if (despuesDelCierre || diaPosterior) {
            throw new IllegalStateException("El plazo de
reservas ha cerrado");
        }
        if (cantidadSolicitada <= 0) {
            throw new IllegalArgumentException("La cantidad
debe ser positiva");
        }
        if (cantidadSolicitada > cantidadDisponible) {
            throw new IllegalStateException("Disponibilidad
insuficiente");
        }
        return cantidadDisponible - cantidadSolicitada;
    }
}
```

5. Fase de pruebas

Los casos de prueba siguientes cubren caminos válidos, inválidos, de límite, autorización y rendimiento. La ejecución real debe realizarse en el entorno de desarrollo y sus resultados deben

adjuntarse como evidencia; este documento presenta el diseño de pruebas, no afirma que hayan sido ejecutadas.

Tabla 9

Casos de prueba del sistema

Caso	Requisito	Entrada / condición	Resultado esperado	Tipo
CP-01	RF-02	Productor autenticado; producto, cantidad 20, precio positivo y unidad válidos.	Oferta creada y visible en catálogo.	Funcional positivo
CP-02	RF-02	Cantidad igual a 0.	Se rechaza; se muestra mensaje y no se guarda oferta.	Límite/negativo
CP-03	RF-04	Reserva el viernes a las 17:59:59.	La solicitud puede continuar si hay disponibilidad.	Límite positivo
CP-04	RF-04	Reserva el viernes a las 18:00:00.	Se rechaza por cierre del plazo.	Límite negativo
CP-05	RF-05	Disponibles 3; cliente solicita 2.	Reserva aceptada; disponibilidad resultante 1.	Funcional positivo
CP-06	RF-05	Disponibles 1; dos solicitudes concurrentes de 1 unidad.	Solo una se confirma; la otra recibe disponibilidad insuficiente; no hay sobreventa.	Concurrencia
CP-07	RF-09 / RNF-02	Cliente intenta abrir reporte de Secretaría.	Acceso denegado y evento registrado.	Seguridad
CP-08	RF-08	Funcionario solicita reporte	Se muestra reporte vacío	Funcional/límite

		de una jornada sin reservas.	con totales cero, sin error del sistema.	
CP-09	RNF-01	Prueba con 100 usuarios concurrentes y datos representativos.	95 % de consultas del catálogo en 3 segundos o menos.	Rendimiento
CP-10	RNF-03	Cinco productores completan publicación después de capacitación.	Al menos cuatro completan la tarea sin ayuda en 20 minutos o menos.	Usabilidad
['CP-11', 'RNF-05', 'Usuario intenta consultar datos personales no necesarios para su rol.', 'La información se limita según permisos y finalidad; no se exponen campos innecesarios.', 'Privacidad/seguridad']				

Nota. Elaboración propia para validar los requisitos funcionales y no funcionales del proyecto.

Tabla 10

Pruebas unitarias propuestas con JUnit 5

Prueba unitaria JUnit	Preparación	Aserción esperada
UT-01: reserva válida	Viernes 17:00; solicita 2; disponibles 5.	El método retorna 3.
UT-02: cantidad no positiva	Solicita 0; disponibles 5.	Lanza IllegalArgumentException.
UT-03: inventario insuficiente	Solicita 3; disponibles 2.	Lanza IllegalStateException.
UT-04: cierre exacto	Viernes 18:00; solicita 1; disponibles 2.	Lanza IllegalStateException.

Nota. Elaboración propia para verificar el funcionamiento de la regla de negocio implementada.

Código de pruebas unitarias JUnit 5

El siguiente ejemplo puede guardarse en src/test/java/ReglaReservaTest.java y ejecutarse con JUnit 5. Debe integrarse con la configuración de dependencias del proyecto antes de reportar resultados.

```
import static org.junit.jupiter.api.Assertions.*;
import java.time.LocalDateTime;
import org.junit.jupiter.api.Test;

class ReglaReservaTest {
    private final ReglaReserva regla = new ReglaReserva();

    @Test
    void reservaValidaDescuentaCantidad() {
        LocalDateTime viernes = LocalDateTime.of(2026, 11, 6,
17, 0);
        assertEquals(3, regla.validarReserva(viernes, 2, 5));
    }

    @Test
    void cantidadCeroEsRechazada() {
        LocalDateTime jueves = LocalDateTime.of(2026, 11, 5,
10, 0);
        assertThrows(IllegalArgumentException.class,
            () -> regla.validarReserva(jueves, 0, 5));
    }

    @Test
    void inventarioInsuficienteEsRechazado() {
        LocalDateTime jueves = LocalDateTime.of(2026, 11, 5,
10, 0);
        assertThrows(IllegalStateException.class,
            () -> regla.validarReserva(jueves, 3, 2));
    }

    @Test
    void viernesALasSeisCierraReservas() {
        LocalDateTime cierre = LocalDateTime.of(2026, 11, 6,
18, 0);
        assertThrows(IllegalStateException.class,
            () -> regla.validarReserva(cierre, 1, 2));
    }
}
```

Matriz de trazabilidad

La matriz conecta los requisitos Must con diseño, componente y pruebas para comprobar que cada requisito imprescindible tiene una forma verificable de validación.

Tabla 11

Matriz de trazabilidad de requisitos

Requisito	Elemento de diseño	Código o componente	Casos de prueba
RF-01	Entidad Productor y formulario de registro	ProductorController / ProductorService	CP-01 (adaptar a registro de productor)
RF-02	Entidad Oferta y pantalla Publicar oferta	OfertaController / OfertaService	CP-01, CP-02
RF-03	Catálogo y consulta de ofertas activas	CatalogoController / OfertaRepository	CP-01 y prueba de catálogo
RF-04	Flujo de reserva y validación del cierre	ReglaReserva.validarReserva()	CP-03, CP-04, UT-04
RF-05	DetalleReserva y operación transaccional	ReservaService / repositorio con bloqueo	CP-05, CP-06, UT-01, UT-03
RF-06	Pantalla y plantilla de confirmación	ConfirmacionReservaDTO	Prueba de reserva confirmada
RF-08	Vista agregada por producto, productor y vereda	ReporteController / ReporteService	CP-08
RF-09	Control de acceso por roles	SecurityConfig / autorización de API	CP-07
RNF-01	Índices de consulta y paginación del catálogo	Prueba de carga / métricas de API	CP-09
RNF-02	Autenticación, roles y almacenamiento seguro de contraseñas	Spring Security y configuración de seguridad	CP-07
RNF-03	Prototipo y evaluación de tareas con usuarios	Interfaz responsiva / protocolo de usabilidad	CP-10
RNF-05	Minimización de datos y control de acceso	Modelo de datos / política de privacidad	CP-11

Nota. Elaboración propia para relacionar requisitos, diseño, componentes y casos de prueba.

6. Fase de despliegue

Entorno de producción. Se recomienda una instancia pequeña de aplicación en la nube, base de datos relacional administrada o protegida, dominio propio, certificado TLS/HTTPS, copias de seguridad automáticas y monitoreo básico. El proveedor y la región de alojamiento deben seleccionarse después de revisar costos, soporte, seguridad, disponibilidad y condiciones de tratamiento de datos.

Estimación mensual orientativa: servidor de aplicación, $80.000–$180.000 COP; base de datos o almacenamiento, $60.000–$150.000 COP; dominio prorrateado, $5.000–$15.000 COP; copias de seguridad y monitoreo, $20.000–$60.000 COP. Total estimado: $165.000–$405.000 COP mensuales, sin incluir impuestos, soporte especializado ni variaciones de proveedor. Estos valores son supuestos de planeación, no cotizaciones verificadas.

Estrategia de lanzamiento: (1) limpiar y validar el listado inicial de productores y productos; (2) hacer una prueba técnica interna; (3) realizar piloto con 10 productores y un grupo pequeño de clientes; (4) registrar incidencias y medir tasa de reserva completada, errores y facilidad de uso; (5) corregir bloqueadores; (6) capacitar al resto de productores con una guía ilustrada y demostraciones; (7) lanzar la plataforma y disponer de soporte durante la primera jornada.

Capacitación. Se recomienda una sesión presencial breve con el celular del productor, una guía de una página con capturas y pasos numerados, ejercicios de publicar y editar una oferta y un canal de soporte. Para participantes con poca experiencia digital, la práctica guiada es más adecuada que una explicación técnica extensa.

Datos iniciales. Si existen hojas de cálculo, se revisarán duplicados y formatos antes de importarlas. La migración se realizará solo para los datos necesarios, con validación y respaldo previo. Si no existen registros estructurados, se realizará el registro inicial asistido.

Tabla 12

Estimación mensual de costos del entorno de producción

Concepto	Estimación mensual COP	Observación
Servidor de aplicación	$80.000–$180.000	Instancia pequeña; depende de proveedor y uso.
Base de datos/almacenamiento	$60.000–$150.000	Servicio administrado o instancia protegida.
Dominio (prorrateado)	$5.000–$15.000	Costo anual dividido por doce; confirmar tarifa.
Copias de seguridad y monitoreo	$20.000–$60.000	Según retención y volumen.
Total orientativo	$165.000–$405.000	No constituye cotización comercial.

Nota. Elaboración propia con estimaciones orientativas para el despliegue del sistema.

Borrador de acta de entrega

Proyecto: Mercado Campesino Digital de Yopal.

Fecha: [completar al momento de la entrega]. Responsable receptor: Secretaría de Desarrollo Económico de Yopal (caso académico ficticio). Equipo entregante: equipo de desarrollo de LlanoSoft S.A.S. (empresa ficticia del taller).

Objeto de entrega: versión inicial de la plataforma web responsiva para publicación de ofertas, reserva previa, consulta de reservas y reportes por producto, productor y vereda.

Entregables previstos: código fuente y README; documentación de arquitectura y modelo de datos; pruebas y evidencias; manual de usuario; configuración de despliegue; inventario de incidencias conocidas; guía de respaldo y restauración.

Criterios de aceptación: los requisitos Must acordados cuentan con pruebas; no existen defectos críticos abiertos; las cuentas y permisos fueron verificados; se ejecutó el piloto y se documentaron sus resultados; se realizó capacitación y se entregaron instrucciones de soporte.

Pendientes y observaciones: [registrar incidencias, responsable y fecha compromiso]. La solicitud de domicilios queda fuera del alcance de esta entrega y requiere evaluación y aprobación independiente.

Firmas: representante de la entidad receptora _____; líder del equipo de desarrollo _____; fecha _____.

7. Fase de mantenimiento

Correctivo. Si el reporte presenta cantidades duplicadas por un error de consulta, se reproduce el problema, se corrige la consulta, se agregan pruebas de regresión y se publica una versión controlada.

Adaptativo. Si cambia una norma aplicable o el formato requerido por la Secretaría, se evalúa el impacto, se actualizan reglas, avisos o reportes y se verifica el cumplimiento antes de desplegar.

Perfectivo. Si los productores solicitan filtros por categoría o una vista más clara de reservas, se prioriza la mejora con usuarios y se incorpora a una iteración futura.

Preventivo. Si una dependencia presenta una vulnerabilidad, se actualiza de forma controlada, se ejecutan pruebas de seguridad y regresión y se documenta el cambio.

Solicitud de domicilios en 2027. La petición se registrará como cambio de alcance. Primero se analizarán necesidades y restricciones (zonas de cobertura, costos, responsables, horarios, tratamiento de direcciones y posibles aliados); luego se estimarán presupuesto y plazo y se solicitará aprobación. Si se aprueba, recorrerá análisis de requisitos, diseño de logística y datos, implementación, pruebas funcionales y de seguridad, despliegue gradual y mantenimiento. No debe añadirse de manera informal a la versión inicial.

Conclusiones

El modelo incremental se ajusta al caso porque permite entregar valor en partes y validar tempranamente la experiencia de productores y clientes. La restricción más determinante es el plazo de cuatro meses combinado con un equipo de tres desarrolladores; por ello, la prioridad debe mantenerse en catálogo, ofertas, reservas y reportes, dejando los domicilios fuera del alcance inicial.

La calidad de la solución depende de requisitos medibles, validaciones del lado del servidor, control transaccional de inventario, pruebas de aceptación y trazabilidad. La facilidad de uso no debe suponerse: debe comprobarse con productores reales mediante tareas observables y métricas.

Finalmente, el tratamiento de datos personales, la seguridad, la capacitación y el mantenimiento deben planearse desde el inicio. El despliegue no representa el final del SDLC, sino el comienzo de la operación, el soporte y la mejora continua.

Referencias

Congreso de Colombia. (2012, 17 de octubre). Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales. Diario Oficial No. 48.587. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

International Organization for Standardization. (2023). ISO/IEC 25010:2023 Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Product quality model. https://www.iso.org/standard/78176.html

Corporación Universitaria Remington. (2026). Taller: Ciclo de vida del desarrollo de software (SDLC) [Material de clase]. Ingeniería de Software, Sede Yopal.